

I-POP: Ignite Positive Prefetchers

Yiquan Lin^{†‡}, Wenhai Lin[‡], Yiquan Chen[‡], Jiexiong Xu^{†‡},
Shishun Cai[‡], Jiarong Ye[†], Zonghui Wang^{†*}, Wenzhi Chen^{†*}

[†]Zhejiang University, [‡]Alibaba Cloud Computing
{linyiquan, zhwang, chenwz}@zju.edu.cn

Abstract—Hardware prefetching is a well-established technique for bridging the processor-memory performance gap. To improve cache miss coverage, modern processors often integrate multiple prefetchers. However, multi-prefetcher systems without proper management often suffer from suboptimal performance due to a surge of useless prefetches. Several techniques have been proposed to select appropriate prefetchers for issuing requests, but they all face limitations. Specifically, existing (1) static schemes lack feedback regulation mechanisms and suffer from inflexible prefetcher selections; (2) reinforcement learning (RL)-based schemes incur high overhead and suffer from adjustment lag; and (3) performance-counter-based schemes rely on inefficient runtime metrics that fail to accurately and clearly reflect a prefetcher’s true impact on performance.

In this paper, we propose I-POP, a high-performance and low-overhead prefetcher management scheme for multi-prefetcher systems. I-POP introduces a novel runtime metric, Prefetch Effectiveness (PE), which aggregates each prefetch request’s beneficial and harmful effects to precisely quantify the impact of a prefetcher on performance, effectively overcoming the limitations of prior metrics. To compute and leverage this metric, I-POP incorporates two key components: the Metric Collector, which periodically calculates each prefetcher’s PE, and the Control Engine, which dynamically manages all prefetchers based on their PE values. Specifically, I-POP ignites (enables) prefetchers with positive PE values, adaptively tuning their aggressiveness, and disables those with non-positive PE. We evaluated I-POP on numerous workloads, and the results show I-POP outperforms two state-of-the-art approaches, Bandit and Aleto, by 4.2% and 3.5% across three benchmark suites in a single-core system, and 6.6% and 8.6% in a 16-core system, while incurring only 1.46 KB of storage overhead.

I. INTRODUCTION

Hardware prefetching [7, 10, 11, 14, 15, 17, 19, 23, 24, 26, 28, 29, 33, 36, 37, 39, 42, 46, 48, 49, 51, 54, 56, 57] is a well-studied technique aimed at bridging the processor-memory performance gap. By leveraging dedicated hardware to detect recurring memory access patterns and preloading data before it is requested, prefetching reduces cache misses and raises instructions per cycle (IPC). A hardware prefetcher is inherently tied to specific memory access patterns it can recognize [29, 31]. When encountering patterns that fall outside its capability, it may fail to provide benefits and, in some cases, even degrade performance. To identify diverse memory access behaviors, researchers have explored various types of prefetchers, including delta-based [14, 19, 28, 36, 37, 46, 48, 49], bit-pattern-based [11, 15, 24, 33, 51], and correlation-based approaches [7, 10, 23, 26, 39, 54, 56, 57], among others.

Modern processors, such as Intel Atom Gracemont [6], AMD Zen3 [4], ARM Neoverse V2 [5], typically integrate multiple types of prefetchers to work simultaneously, broadening the range of recognizable patterns and thus improving cache miss coverage. Nevertheless, it also amplifies the negative side effects of useless prefetches, significantly inflating memory bandwidth consumption and potentially degrading overall performance.

Therefore, it is crucial to employ multi-prefetcher management techniques that selectively activate prefetchers for issuing requests. We evaluate a multi-prefetcher system with and without management in Section II-A. As shown in Figure 1, an unmanaged multi-prefetcher system can incur performance losses of up to 24.3%. In contrast, effective prefetcher management can deliver performance improvements of up to 22.6% over unmanaged systems.

Existing multi-prefetcher management techniques [20, 29, 31, 42, 46, 58] can be categorized into static [29, 42] and dynamic schemes. The latter can be further divided into RL-based approaches [20, 58] and performance-counter-based approaches [31, 46]. However, all existing approaches exhibit notable limitations.

Static schemes [29, 42] naively employ static strategies to manage multiple prefetchers for simplicity. These methods assign a predefined priority to each prefetcher and accept predictions only from the one with the highest priority among those that generate predictions. However, such static approaches lack feedback regulation mechanisms, exacerbating performance degradation when prefetching becomes detrimental. Furthermore, their decision-making process is inherently inflexible, as it rigidly adheres to a fixed priority order, even when lower-priority prefetchers demonstrate superior predictions [31].

RL-based schemes [20, 58] use online RL techniques to guide prefetcher management. However, they incur significant computation and storage overhead, as they require maintaining action values and transition probabilities for each unique state of the environment, where the state space is extremely large. Although Bandit [20] leverages an RL algorithm without state transitions, its action space grows exponentially and still involves computationally expensive square root and logarithmic operations. Moreover, Bandit suffers from adjustment lag. When access patterns change, it fails to detect and adjust its strategy promptly. Instead, it undergoes a prolonged adaptation process until the reward of the better strategy surpasses that of the current one, which degrades performance.

Performance-counter-based schemes [31, 46] rely on

* Corresponding Author

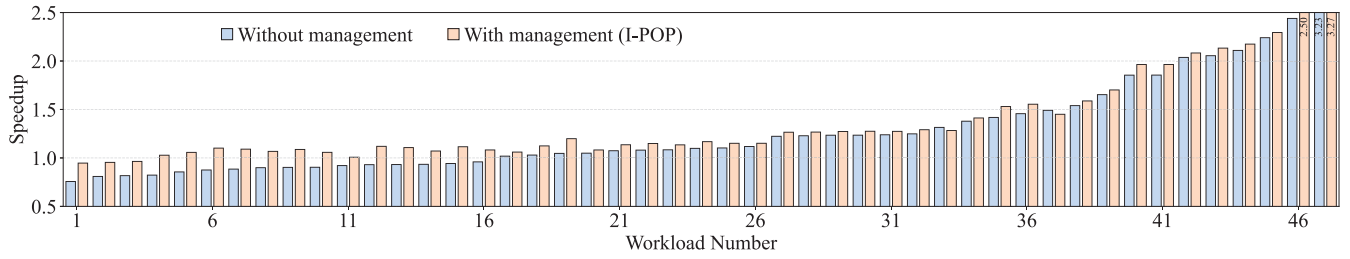


Fig. 1: IPC speedup of multi-prefetcher systems with and without prefetcher management.

specific runtime metrics (e.g., prefetch accuracy) to guide prefetcher management. They continuously monitor each prefetcher’s metrics and adjust strategies accordingly. Specifically, Sandbox [46] selects the top-K prefetchers based on their metric values, while Aleto [31] determines strategies by comparing each prefetcher’s metric (accuracy) against fixed thresholds. However, both approaches are limited by the inefficiency of their chosen metrics. First, these metrics exhibit low, and sometimes even negative, correlation with IPC performance, thereby failing to accurately reflect a prefetcher’s true impact on IPC performance. Second, their metrics fail to offer a clear and intuitive criterion for prefetcher management, relying on top-K selections or comparisons to fixed thresholds. Nevertheless, the optimal K and proper thresholds vary significantly across different workloads and DRAM bandwidth conditions. Therefore, they often suffer from suboptimal performance.

In this paper, we propose I-POP, a high-performance, low-overhead prefetcher management scheme for multi-prefetcher systems. To address the inefficiency of prior single-dimensional evaluation metrics, I-POP introduces a novel runtime metric, Prefetch Effectiveness (PE), which offers a comprehensive assessment of prefetching by capturing both positive and negative impacts. Specifically, prefetching improves performance by reducing cache misses through useful prefetches, but it can also degrade performance in two major ways: (1) cache pollution that evicts useful data and induces additional misses, and (2) increased latency of demand accesses due to heightened contention for shared physical resources. PE accurately quantifies a prefetcher’s net impact on performance by aggregating these three effects into a single, interpretable score. Crucially, PE exhibits a strong correlation with IPC (see Figure 3) and provides an intuitive control signal: a positive PE indicates benefits, whereas a negative PE signifies harms.

To compute and leverage PE effectively, I-POP employs two main components: Metric Collector and Control Engine. The Metric Collector continuously tracks beneficial and harmful effects of prefetch requests, periodically calculates each prefetcher’s PE, and commits the results to the Control Engine. Based on the PE values, the Control Engine dynamically manages each prefetcher’s on/off status and aggressiveness levels. It *ignites* prefetchers with positive PE values, keeping them active and adaptively tunes their aggressiveness based on

average DRAM latency and other prefetchers’ PE. Prefetchers with non-positive PE values are promptly disabled. When the DRAM bandwidth is relatively abundant, the Control Engine reactivates disabled prefetchers, allowing them another opportunity to contribute.

Result summary. We implemented I-POP on Champ-sim [21] simulator and evaluated it on 76 single-core workloads, spanning SPEC17 [1], GAP [13], and Ligra [50]. Results show that PE exhibits positive correlations with IPC performance on 99% of workloads and achieves an average correlation of 0.85, exceeding the best prior metric by 0.30. Additionally, I-POP improves performance by 9.7% over a baseline that uses a PC-Stride [19] prefetcher with basic throttling. Compared to two state-of-the-art schemes, Bandit and Aleto, I-POP delivers 4.2% and 3.5% higher performance in single-core systems across three benchmark suites, and 6.6% and 8.6% higher performance in 16-core systems, respectively. Moreover, it incurs only 1.46 KB of storage overhead, which is 27.7% of Aleto’s requirement.

We make the following contributions in this paper:

- We thoroughly analyze existing prefetcher management schemes and pinpoint why they suffer from suboptimal performance.
- We propose a novel runtime metric, PE, which accurately and clearly reflects the effect of a prefetcher on IPC performance by aggregating the beneficial and harmful effects of each prefetch request.
- We design a high-performance and low-overhead prefetcher management scheme for multi-prefetcher systems, called I-POP. I-POP periodically computes all prefetchers’ PE values and uses these values to guide dynamic adjustments of their on/off status and aggressiveness levels.
- We implement I-POP and comprehensively evaluate it across a wide range of workloads and system configurations. Results show that I-POP outperforms prior state-of-the-art approaches.

II. BACKGROUND AND MOTIVATION

A. Why Is Prefetcher Management Necessary?

Deploying multiple prefetchers can cover a broader range of cache misses than a single prefetcher [31, 58]. However, this also introduces a higher volume of useless prefetches,

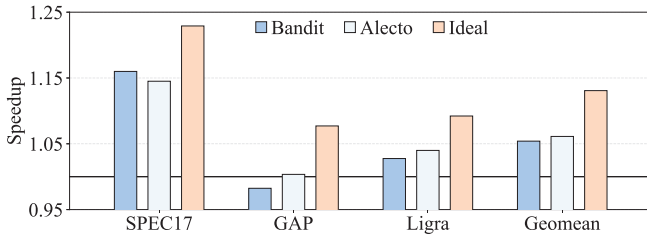


Fig. 2: Comparison of IPC speedup among state-of-the-art schemes and the ideal, where L2 cache is equipped with Berti, BOP, PMP, and Prophet.

leading to excessive DRAM bandwidth consumption, which can ultimately degrade performance.

Figure 1 compares the performance of a multi-prefetcher system with and without management, where L2 cache is equipped with Berti [37], BOP [36], PMP [24], and Prophet [30]. We only show workloads with performance differences greater than 3%. Under these workloads, unmanaged prefetching, where all prefetchers issue requests indiscriminately, degrades performance by up to 24.3% compared to the baseline. These degradations stem from DRAM traffic surging to 2.86 $\times$ the baseline level, overwhelming system resources and introducing severe contention. In contrast, effective prefetcher management mitigates the negative side effects of prefetching, delivering up to 22.6% performance improvement over unmanaged prefetching. These results highlight the necessity of prefetcher management to fully realize the benefits of multi-prefetcher systems.

B. Why Not Static Schemes?

For simplicity, DOL [29] and IPCP [42] adopt static strategies, which prioritize prefetchers based on a fixed order (e.g., $P_1 > P_2 > P_3 > P_4$) predefined at design time. Specifically, when a demand access occurs, DOL sequentially propagates its information through all prefetchers. Once a prefetcher generates a prediction, DOL immediately adopts it and terminates further propagation. In contrast, IPCP broadcasts information to all prefetchers simultaneously but only accepts results from the highest-priority one among those generating predictions.

While straightforward to implement, static prefetcher management suffers from two key limitations: (1) Lack of feedback regulation mechanisms. Prefetchers can sometimes degrade performance, while static management lacks mechanisms to detect and mitigate negative impacts. They continue enforcing predefined rules, exacerbating performance degradation. (2) Inflexible prefetcher selection. Consider a program counter (PC) exhibiting a common stride access pattern: $(+2, +2, +2, +6)$. A PC-Stride[19] prefetcher (P_2) would confidently recognize it as a pattern with $+2$ stride, whereas an SMS [51] prefetcher (P_3) could fully capture the true pattern and provide superior predictions. Nevertheless, due to rigid priority rules, P_2 would always be favored over P_3 , leading to suboptimal performance. These limitations highlight the need for more adaptive and intelligent prefetcher management.

C. Are Existing Dynamic Schemes Effective Enough?

Several dynamic management techniques [20, 31, 46, 58] have been proposed for multi-prefetcher systems, which can be broadly classified into RL-based [20, 58] and performance-counter-based [31, 46] approaches. However, both remain with significant room for improvement.

To quantify the potential gains, we model a phase-level ideal prefetcher management scheme, i.e., using a fixed prefetcher combination during each phase (1K demand accesses), since achieving perfect management at the per-demand level is impractical. We track the IPC of all possible prefetcher combinations and, for each phase, select the combination that yields the highest IPC to construct the ideal scheme. Figure 2 compares the performance of two state-of-the-art approaches (Bandit and Alecto) against the ideal. On average, Bandit and Alecto fall short of the ideal by 7.7% and 7.0%, respectively. Alecto shows its largest performance headroom on SPEC17, with up to 8.4% unrealized performance. Bandit’s largest gap appears on GAP, reaching 9.5%, where its ineffective management of multiple prefetchers even causes it to underperform the single-prefetcher baseline.

D. Why Do RL-based Schemes Struggle?

RL-based schemes [20, 58] utilize RL techniques to guide prefetcher management, with Bandit [20] being the most representative approach. Bandit employs an online learning strategy to explore various prefetcher combinations at phase granularity (every 1000 demand accesses). A prefetcher combination represents the collective state of all prefetchers. For instance, in a system with four prefetchers, $(P_1 - ON, P_2 - ON, P_3 - OFF, P_4 - OFF)$ is one possible combination. At the beginning of each phase, Bandit evaluates each combination and selects one to adopt. After the phase ends, the observed IPC is assigned as the reward for the selected combination. Combinations with higher average rewards or fewer past selections are prioritized in subsequent phases.

A key limitation of Bandit is its adjustment lag. Commonly, when memory access patterns shift or DRAM bandwidth contention changes, an alternative prefetcher combination may yield better performance than the current one. However, Bandit fails to switch to this better combination in time. Instead, it follows a prolonged adaptation process. It keeps suboptimal strategies until the better combination’s occurrences are sufficiently low relative to the total selection count. Only after the better combination’s average reward (average observed IPC) is high enough does Bandit adopt it steadily. Bandit’s delayed adjustments result in performance losses, leading to a 6.4% performance gap compared to the ideal management.

Moreover, Bandit suffers from significant computation overhead. Its algorithm must evaluate all possible prefetcher combinations during each phase. Specifically, in a system with N prefetchers, each supporting S possible states, the total number of distinct combinations grows exponentially as S^N . Furthermore, evaluating each combination requires computationally expensive operations, including square roots and logarithms.

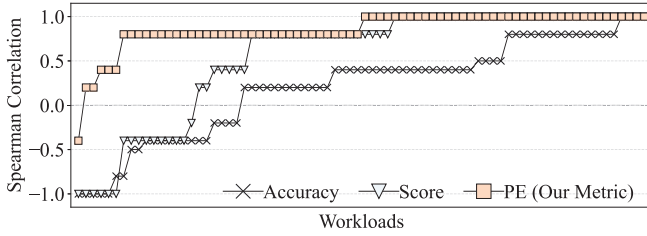


Fig. 3: Distribution of Spearman's rank correlation coefficient between each metric and IPC performance across all single-core workloads, sorted in ascending order.

E. Why Do Existing Performance-counter-based Schemes Fall Short?

Existing performance-counter-based schemes [31, 46], including Sandbox [46] and Alecto [31], adopt specific runtime metrics as proxies for performance, such as prefetch accuracy. They continuously monitor each prefetcher's metrics and periodically adjust management strategies based on the observed values. Specifically, Sandbox selects the top- K prefetchers based on their metric values and disables the rest, while Alecto compares each prefetcher's metric value against fixed thresholds to determine its strategy. In the following, we first detail both approaches and then analyze their limitations.

1) **Sandbox**: Sandbox employs a runtime metric called score, which increments for each accurate prefetch, to guide prefetcher management at phase granularity. During each phase, it evaluates each prefetcher in a round-robin fashion by issuing pseudo-prefetch requests to a bloom filter and collecting the corresponding scores. At the end of the phase, it selects the top- K scoring prefetchers to issue real prefetches in the next phase.

2) **Alecto**: Alecto adopts prefetch accuracy as the proxy for each prefetcher's performance, where accuracy is defined as the fraction of issued prefetches that are actually required by the program. To determine which prefetchers to activate, Alecto periodically compares each prefetcher's accuracy value against two fixed thresholds: the Deficiency Boundary (DB) and the Proficiency Boundary (PB). If a prefetcher's accuracy falls below DB, it is turned off. Conversely, if the accuracy exceeds PB, Alecto increases the prefetcher's aggressiveness while disabling prefetchers with accuracy below PB.

3) **Limitation**: Both metrics they rely on are inefficient, leading to suboptimal performance.

First, both metrics fail to accurately reflect a prefetcher's true impact on IPC performance. To evaluate this, we analyze the correlation between each metric and IPC using Spearman's rank correlation coefficient (ρ). A positive (or negative) ρ indicates that higher metric values are associated with increased (or decreased) IPC. The absolute value of ρ further reflects the strength of the monotonic relationship: the larger the magnitude, the stronger the association.

For each workload and metric, we conduct evaluations of IPC performance and corresponding metric values for each of the four prefetchers, and then compute ρ using these four pairs

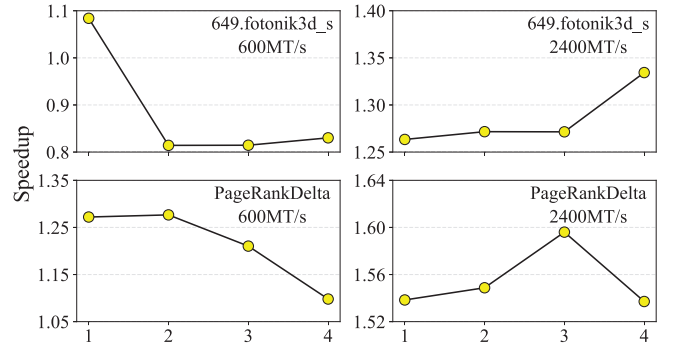


Fig. 4: IPC speedup of Sandbox with varying K under different DRAM bandwidths, normalized to no prefetching.

of metric values and IPC performances. Figure 3 presents the distribution of ρ between each metric and IPC performance across all single-core workloads. On average, ρ for accuracy and score metrics is only 0.21 and 0.55, respectively. Notably, accuracy and score metric fail to exhibit positive correlation on 29% and 21% of workloads, respectively, and even exhibit complete anticorrelation ($\rho = -1$) with IPC performance in some cases. These results reveal that neither accuracy nor score accurately reflects the true impact of prefetching on IPC performance. Therefore, relying on such metrics results in suboptimal prefetcher management, ultimately leading to performance degradation.

Second, their metrics fail to offer a clear and intuitive criterion for prefetcher management. For example, if a prefetcher exhibits an accuracy of 20%, it remains ambiguous whether it improves or degrades performance. Although Sandbox and Alecto attempt to address this issue by selecting the top- K prefetchers or comparing to thresholds, their effectiveness is limited, as proper K and thresholds vary significantly across different workloads and DRAM bandwidth conditions.

K vs. Workload, Bandwidth: Figure 4 illustrates the performance of Sandbox with varying K values on '649.fotonik3d_s' and 'PageRankDelta', demonstrating that the best-performing K differs between applications. At 2400 MT/s, the optimal K for these two workloads is 4 and 3, respectively. Furthermore, fluctuations in DRAM bandwidth can also impact the optimal K . For example, in '649.fotonik3d_s', the best K values at 600 MT/s and 2400 MT/s are 1 and 4, respectively. These variations occur because increasing K allows more prefetchers to operate simultaneously, covering a larger portion of cache misses. However, a large K also introduces a surge of useless prefetches, degrading performance under limited DRAM bandwidths. Thus, using a fixed K value across all scenarios usually leads to suboptimal prefetcher management.

Threshold vs. Workload: Figure 5 shows the speedup and accuracy of SMS prefetcher on 6 workloads under the same system configuration. In some cases (left half), low accuracies ($< 20\%$) still deliver performance improvement up to 18%. However, in other cases (right half), even higher accuracies

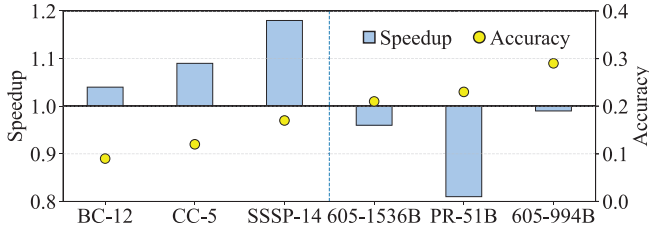


Fig. 5: IPC speedup and accuracy of SMS prefetcher on six workloads, normalized to no prefetching.

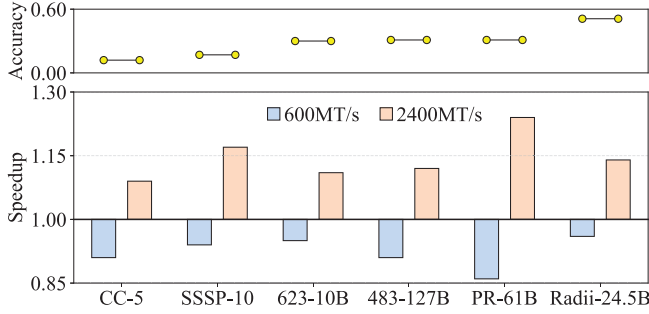


Fig. 6: IPC speedup and accuracy of SMS prefetcher under varying DRAM bandwidths, normalized to no prefetching.

(> 20%) correlate with performance degradation of up to 19%. These reversals indicate that no fixed accuracy threshold can consistently determine a prefetcher’s impact across all workloads.

Threshold vs. Bandwidth: Figure 6 illustrates the speedup and accuracy of SMS prefetcher under two different DRAM bandwidth configurations (600 MT/s and 2400 MT/s). We make two key observations. First, the accuracy remains unchanged when only DRAM bandwidth changes. This is expected, as the code and data remain the same, leading prefetchers to generate identical predictions. Second, the performance differs drastically between the two bandwidth conditions despite constant accuracy. At 600 MT/s, all cases in Figure 6 exhibit performance degradation, with a performance loss of up to 38% compared to results at 2400 MT/s. Given that available DRAM bandwidth fluctuates dynamically in real-world multi-core systems, relying on fixed accuracy thresholds for prefetcher management often results in suboptimal decisions.

Furthermore, we evaluate an FDP-inspired mechanism [53], a qualitative design that simultaneously considers accuracy, cache pollution, and DRAM bandwidth contention¹ as runtime metrics and compares them against fixed thresholds. Figure 7 shows its performance relative to a quantitative design, which uses the same event counts but adopts a quantitative decision-making methodology. Although the FDP-inspired mechanism incorporates multi-dimensional metrics, its qualitative threshold-based decisions still lead to suboptimal

¹Pollution = $\frac{N_{LLC}^{POLL} + N_{DRAM}^{POLL}}{\text{number of demand misses}}$; bandwidth contention = $\frac{N_{Bus} + N_{Bank}}{\text{number of prefetches}}$. Notations are detailed in Table I.

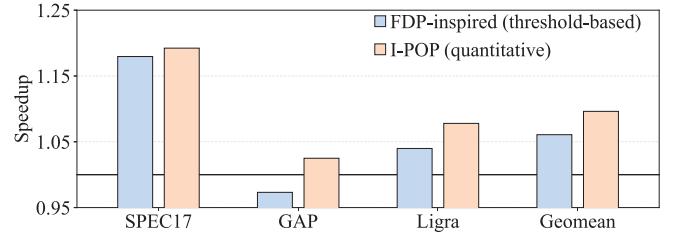


Fig. 7: IPC speedup of FDP-inspired (threshold-based) and I-POP (quantitative) mechanisms

TABLE I: Notation summary

I_{UFP}, I_{POLL} I_{LAT}	Impact of useful prefetches, cache pollution, and increased access latency.
$N_{LLC}^{UPF}, N_{DRAM}^{UPF}$	Number of useful prefetches served by the LLC and DRAM.
$N_{LLC}^{POLL}, N_{DRAM}^{POLL}$	Number of pollution-induced misses served by the LLC and DRAM.
N_{Delay}, N_{Bus} N_{Bank}	Number of on-chip interconnect contention, DRAM bus contention, and bank conflict events.
L_{LLC}, L_{DRAM}	Average miss latency for requests served by the LLC and DRAM.
T_{NoC}, T_{Bus} T_{Bank}	Time during which a prefetch occupies shared on-chip interconnects, DRAM buses, and banks.
$T_{Command}, T_{Data}$	Latency of transmitting a DRAM command and returning its associated data burst.
$\overline{RB}_{miss}$	Average row buffer miss rate across all DRAM channels, updated periodically at runtime.

performance. In contrast, using the same metrics under a quantitative decision methodology delivers 3.5% higher performance on average, which motivates us to design a quantitative analysis-based mechanism.

III. I-POP: RUNTIME METRIC

Section II demonstrates two key limitations of existing metrics: (1) they exhibit low correlation with IPC, thus failing to accurately reflect a prefetcher’s true impact on performance; and (2) they lack a clear and intuitive criterion for guiding prefetcher management. To address these issues, we propose Prefetch Effectiveness (PE) in this section, a novel runtime metric designed to precisely quantify the effect of a prefetcher on IPC performance. Table I summarizes the notations used and their descriptions.

A. Overall

Unlike prior single-dimensional evaluation metrics, PE offers a comprehensive assessment by capturing both the positive and negative impacts of prefetch requests on IPC performance. In principle, (1) the benefit of prefetching lies in its ability to reduce cache misses via useful prefetches. However, prefetching can also degrade performance through: (2) cache pollution that induces additional misses, and (3) increased latency of demand accesses due to intensified contention for shared hardware resources. Based on this principle, we aggregate these three contributing factors to compute PE, as shown in Equation 1. The following subsections detail how we compute

these three components, assuming prefetchers are deployed at L2 cache.

$$PE = I_{UPF} - I_{POLL} - I_{LAT} \quad (1)$$

B. Gains from Useful Prefetches

Useful prefetches can eliminate cache misses, thereby reducing request latency and improving overall system performance. Thus, I_{UPF} can be computed by accumulating the latency reduction brought by every useful prefetch. Specifically, the latency reduction from a useful prefetch is approximately equal to its own access latency, i.e., the time interval between the start of processing this prefetch and the return of requested data. However, this method incurs considerable storage overhead. The reason is that the latency of each prefetch must be saved until it is determined to be useful or not, and then either added to I_{UPF} or discarded. Since the usefulness of a prefetch is often revealed only after a long delay, the system must maintain latency records for numerous undetermined prefetches over an extended period, resulting in high storage costs.

To mitigate this overhead, we propose an approximate estimation method, as shown in Equation 2. The distinction between useful prefetches served by the last-level cache (LLC) and DRAM is based on the observation that their access latencies differ significantly.

$$I_{UPF} = N_{LLC}^{UPF} \cdot L_{LLC} + N_{DRAM}^{UPF} \cdot L_{DRAM} \quad (2)$$

Our method shares similarities with Berto [38], an extension of Berti [37]. Both utilize average latency as an estimation. Berto employs an exponentially weighted moving average to emphasize the latency of recent accesses. In contrast, we adopt a simpler arithmetic mean, which assigns equal weight to the latency of all accesses throughout the entire phase.

C. Losses from Cache Pollution

Cache pollution occurs when useful data is prematurely evicted from the cache to accommodate prefetched data, introducing additional cache misses that would not have occurred otherwise. Our definition of pollution events follows the same principle as FDP [53]. However, FDP estimates the pollution impact by using the ratio of pollution events to total demand misses, whereas PE quantifies the extra latency cost incurred by pollution. We define I_{POLL} as:

$$I_{POLL} = N_{LLC}^{POLL} \cdot L_{LLC} + N_{DRAM}^{POLL} \cdot L_{DRAM} \quad (3)$$

In inclusive cache hierarchies, most pollution-induced misses are likely to be served by the LLC, allowing Equation 3 to be simplified accordingly.

Although prior work such as DSPatch [15] reported that pollution is negligible in large LLCs, its impact increases as cache capacity decreases, particularly in the L1 cache. In our evaluated configurations and workloads, I_{POLL} contributes 26.9% of the total negative effects ($I_{POLL} + I_{LAT}$) on average. Therefore, PE explicitly models cache pollution to provide more accurate estimates.

D. Losses from Increased Access Latency

Prefetches compete with demand requests for physical resources, which increases the demand access latency and ultimately degrades performance. A straightforward way to estimate I_{LAT} is to compare the access latency between cases with and without prefetching. However, this is impractical in real systems, since the access latency without prefetching cannot be directly measured once prefetching is enabled.

We propose a novel method to compute I_{LAT} by accumulating the delay each prefetch request imposes on in-flight demand requests. Specifically, we estimate the delay induced by a prefetch only when its data returns and there are still in-flight demand requests residing in the Miss Status Holding Register (MSHR). This condition ensures that the prefetch actually interferes with pending demand traffic. Furthermore, if a single prefetch affects multiple demand requests, we accumulate only the maximum induced delay.

We model the delay a prefetch imposes on a demand request as the time during which physical resources required by both them are occupied by the prefetch, excluding queuing delays. This reflects the actual service time, which tends to be stable for a given resource and request. The rationale is that, had the prefetch not been issued, the demand request would likely have experienced less contention and been serviced earlier on the shared resources, even if there is no direct overlap or simultaneous contention between the two requests. Our model focuses on three key contention points:

(1) On-chip interconnect contention. When multiple memory requests traverse shared physical links or routers in the on-chip interconnect, they contend for these shared resources. In systems employing a mesh-based Network-on-Chip, T_{NoC} is estimated as the product of the minimum number of hops (routers) a request traverses and the per-hop latency. For systems with other Network-on-Chip topologies (e.g., ring, crossbar, hierarchical mesh), the estimation of T_{NoC} requires careful modeling based on architecture-specific parameters and interconnect characteristics.

(2) DRAM command and data bus contention. If a prefetch request and a demand request are mapped to the same memory channel, they contend for the DRAM command and data buses. Typically, T_{Bus} comprises one data burst and one to three command transmissions, depending on whether the request results in a row hit, a closed row, or a row conflict. We estimate T_{Bus} as $(1 + 2 \cdot \overline{RB}_{Miss}) \cdot T_{Command} + T_{Data}$.

(3) Bank conflict. If a prefetch request and a demand request target the same DRAM bank, the bank cannot serve the demand request until it has completed processing the prefetch request. We estimate T_{Bank} as $t_{CAS} + \overline{RB}_{miss} \cdot (t_{RP} + t_{RCD})$, where t_{CAS} , t_{RP} , and t_{RCD} are standard DRAM timing parameters.

Aggregating the three types of resource contention described above, the final formulation of I_{LAT} is shown in Equation 4. Specific values used in this paper are detailed in Table III.

$$I_{LAT} = N_{Delay} \cdot T_{NoC} + N_{Bus} \cdot T_{Bus} + N_{Bank} \cdot T_{Bank} \quad (4)$$

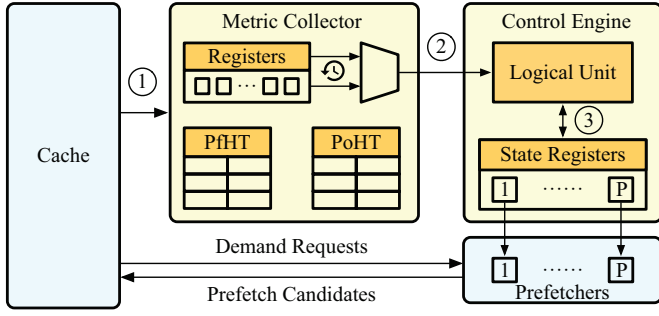


Fig. 8: Overview of I-POP

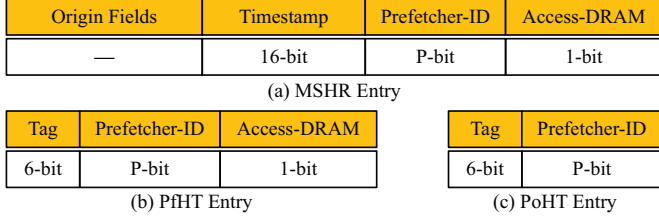


Fig. 9: Entry structures of (a) MSHR, (b) Prefetch Holding Table, (c) Pollution Holding Table.

E. Advantages over Prior Metrics

(1) PE more accurately captures the performance impact of prefetching. Benefiting from a thorough consideration of impact factors, PE exhibits positive correlations with IPC speedup on 99% of workloads and reaches an average correlation of 0.85, outperforming the best prior metric by 0.30, as shown in Figure 3. (2) PE provides a clear and intuitive criterion for prefetcher management: a positive PE ($PE > 0$) indicates a beneficial effect, whereas a non-positive PE ($PE \leq 0$) signals a negative impact. With PE, there is no longer a need to set fixed metric thresholds or use the top-K method for prefetcher management.

IV. I-POP: DESIGN

Figure 8 shows a high-level overview of I-POP, which consists of two key modules: Metric Collector and Control Engine. Metric Collector continuously tracks the information required for PE computation (①), periodically calculates PE per prefetcher, and submits results to the Control Engine (②). Note that PE is computed separately for each prefetcher. To identify the issuing prefetcher(s), every MSHR entry includes a P-bit Prefetch-ID field (1 bit per prefetcher), as shown in Figure 9. Correspondingly, the counters (denoted by N) in the equations in Section III are maintained per prefetcher. The Control Engine dynamically adjusts each prefetcher's state register, which controls its activation and aggressiveness (③). For each prefetcher, on/off decisions are based solely on its own PE, whereas aggressiveness tuning considers both PEs of all prefetchers and L_{DRAM} . In the following, we present a detailed description of them.

A. Metric Collector

The Metric Collector consists of two direct-mapped lookup tables, the Prefetch Holding Table (PfHT) and the Pollution Holding Table (PoHT), along with several counting registers. PfHT and PoHT store information for useful prefetches and cache pollution counting, respectively. These Registers maintain various event counts required for PE computation. We then detail the collection of each variable in PE computation individually.

Latency measuring. Each MSHR entry is extended with a 16-bit timestamp to track a request's latency and a 1-bit Access-DRAM flag to indicate whether the request is served by the LLC or DRAM, as shown in Figure 9(a). Upon allocation, the timestamp is initialized to the current clock cycle. The miss latency of a request is then calculated by subtracting the stored timestamp from the timestamp at data return. Based on the Access-DRAM flag, latencies are accumulated separately for requests served by the LLC and for those served by DRAM. The average miss latencies L_{LLC} and L_{DRAM} are subsequently computed by dividing the accumulated latencies by the number of requests served in each category.

Useful prefetches counting. The Metric Collector tracks the effectiveness of each prefetch request using PfHT, whose entry structure is shown in Figure 9(b). The PfHT is indexed by address, and each entry maintains a 6-bit tag for lookup, a P-bit Prefetch-ID field for identifying prefetchers, and a 1-bit Access-DRAM field for recording whether the request is served by LLC or DRAM.

When prefetched data fills the cache, the Metric Collector inserts relevant metadata of this prefetch into PfHT based on its address. This delayed insertion (at data return rather than issuance) ensures prefetching timeliness and avoids an extra lookup to set the Access-DRAM field compared to inserting at issuance. On each cache hit, the Metric Collector looks up PfHT using the address. If a match is found, it increments the corresponding prefetcher's counter for useful prefetches, either N_{LLC}^{UPF} or N_{DRAM}^{UPF} , and resets the entry.

Cache pollution counting. The detection of cache pollution is managed by PoHT, whose entry structure is shown in Figure 9(c). When a cache eviction occurs due to a prefetch fill, the Prefetch-ID of this prefetch is inserted into PoHT based on the evicted address. For each demand miss, the Metric Collector checks PoHT for the missed address after its data returns. If a match is found, the counter corresponding to the prefetcher identified by the Prefetcher-ID field in the matched PoHT entry, either N_{LLC}^{POLL} or N_{DRAM}^{POLL} , is incremented based on the Access-DRAM field in the associated MSHR entry.

Delay counting. We extend the MSHR to detect in-flight demand requests whenever a prefetch's data returns. If any such demand request exists, the prefetcher that issued the prefetch request increments its N_{Delay} counter. For prefetches that miss in the LLC and access DRAM, we further check whether they map to the same memory channel or bank as any in-flight demand request. If any demand request maps to the same channel as the prefetch, we increment N_{Bus} ; if

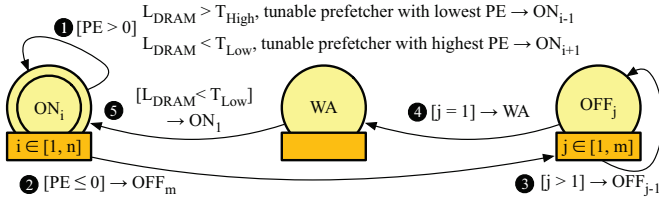


Fig. 10: State transition logic of the Control Engine.

it also maps to the same bank, we increment N_{Bank} . This mapping check is implemented by comparing specific address bits of the prefetch and demand requests, based on the system's memory address decoding, which typically partitions addresses into fields indicating the channel, rank, bank, and others. Since MSHRs already perform block address matching when data returns, I-POP adds only a simple bit-level check, which is low-overhead and practical.

PE calculation. I-POP holds a counter that increments with each demand access. Once it reaches phase length (e.g., 1024), the Metric Collector calculates PE per prefetcher based on equations in Section III. The results are then submitted to Control Engine, and all counters are reset for the next phase.

B. Control Engine

I-POP assigns a state register for each prefetcher, which stores its current state, and the Control Engine controls each prefetcher by modifying its corresponding state register. I-POP defines three state types for every prefetcher: (1) ON_i : Enabled state, with n sub-states representing different levels of aggressiveness, where $i \in [1, n]$. (2) WA : Wait-for-Activation state, where the prefetcher remains disabled until reactivated by the Control Engine. (3) OFF_j : Disabled state, with m sub-states indicating the minimum number of phases the prefetcher must stay off, where $j \in [1, m]$.

After receiving PE values from the Metric Collector, the Control Engine dynamically adjusts each prefetcher's state, as shown in Figure 10.

① If a prefetcher is currently in ON_i and its PE is positive, the Control Engine keeps it active and dynamically adjusts its aggressiveness based on L_{DRAM} , taking into account all tunable prefetchers' PE. Specifically, if L_{DRAM} exceeds a high threshold ($L_{DRAM} > T_{High}$), indicating severe DRAM contentions, the Control Engine reduces the aggressiveness of the tunable prefetcher with the lowest PE by transitioning it from ON_i to ON_{i-1} ($i > 1$). Conversely, if L_{DRAM} is below a low threshold ($L_{DRAM} < T_{Low}$), suggesting low DRAM bandwidth pressure, it increases the aggressiveness of the tunable prefetcher with the highest PE, by transitioning it from ON_i to ON_{i+1} ($i < n$). In our implementation, T_{High} and T_{Low} are set to $3\times$ and $1.5\times$ the ideal DRAM access latency, a system-dependent constant.

② For prefetchers with non-positive PE values, the Control Engine immediately disables them by transitioning their states to OFF_m , preventing further performance degradation.

③ Prefetchers that have been in state OFF_j transition to

TABLE II: Parameters of the simulated system

Core	1-16 cores, 4-wide Out-of-order, 256-entry ROB, 72-entry LQ, 56-entry SQ, Perceptron-based [25]
L1 I/D	Private, I/D split, 64 KB each, 64 B line, 4-way, LRU, 8-entry PQ, 8/16-entry MSHRs, 4-cycle
L2C	Private, 2 MB, 64 B line, 16-way, 32-entry MSHRs, 32-entry PQ, LRU, PC-Stride with 6 degrees, 16-cycle
LLC	Shared, 3 MB/core, 64B line, 12-way, 32-entry PQ, 64-entry MSHRs, SHiP [55], 32-cycle
Network	Only for multi-core systems, 2D mesh topology, each node has a router and an LLC slice, XY routing
DRAM	1C: single channel, 1 rank/channel, 8 banks/rank; 2C, 4C, 8C, 16C: $(cores/2)$ channels, 2 ranks/channel, 8 banks/rank; 4 KB row-buffer/bank; 64-bit data bus; 4800 MTPS; burst length 16; t_{RP} , t_{RCD} , t_{CAS} : 16 ns

OFF_{j-1} at the end of each phase, as long as $j > 1$. ④ Once a prefetcher reaches OFF_1 , the Control Engine moves it to the WA state. ⑤ When $L_{DRAM} < T_{Low}$, indicating relatively abundant DRAM bandwidth, the Control Engine reactivates prefetchers in the WA state by transitioning them to ON_1 .

V. METHODOLOGY

We evaluate I-POP using ChampSim [21]. It is a trace-driven simulator and does not contain wrong-path executions. In real systems, however, mispredicted control flow can cause prefetchers to issue wrong-path prefetches that degrade effectiveness. By design, I-POP's PE metric quantifies effectiveness in the presence of this degradation. Specifically, both I_{UPF} and I_{POLL} capture the impact of wrong-path prefetching, whereas I_{LAT} tends to overemphasize the delay that prefetch requests impose on spurious wrong-path demand requests, biasing the overall PE downward.

System configuration. Table II summarizes the key microarchitectural parameters of our simulated system. The baseline system includes a PC-Stride [19] prefetcher at L2 cache and a basic throttling mechanism that reduces prefetch aggressiveness at 50% and 75% DRAM bandwidth utilization, and drops all prefetches beyond 90%. These threshold values follow those adopted in [9, 15]. Duplicate prefetch requests are merged upon insertion into the prefetch queue (PQ). For multi-core systems, we adopt a sliced LLC architecture and model the on-chip interconnect using a 2D mesh topology. Both single-core and multi-core simulations include a warm-up phase of 50 M instructions, followed by simulation phases of 200 M and 100 M instructions, respectively. Performance is reported as IPC improvement (i.e., speedup) over the baseline system.

Energy Model. We adopt the energy model proposed by Berti [37] to evaluate the dynamic energy consumption of the memory hierarchy. Cache energy is estimated using CACTI-P [32], accounting for tag and data array reads and writes. DRAM energy is obtained using the Micron DRAM Power Calculator [2]. All estimations are based on a 22 nm process technology.

Workloads. We evaluate I-POP across a diverse set of workloads, including SPEC17 [1] (provided by DPC-3 [3]), GAP [13] (provided by Berti [37]), and Ligma [50] (provided

TABLE III: Details of prefetcher management schemes

DOL	Priority orders: Berti > BOP > Prophet > PMP
Bandit	$2^4 = 16$ arms, on/off two states each prefetcher; phase length: 1K L2 demands; the same as [20]
Alecto	$DB = 0.05$, $PB = 0.75$; the same as [31]
Sandbox	$K = 1$; bloom filter size: 2048, 6 hash functions; phase length: 1K L2 demands (256 per prefetcher)
I-POP	PfHT and PoHT: 512-entry; phase length: 1K L2 demands; $T_{NoC} = 0$ (single-core), 2 cycles (multi-core); $T_{Command}, T_{Data} = 1$, 4 DDR cycles; $OFF_j, j \in [1, 3]$; $ON_i, i \in [1, 5]$; $degree = 2 \cdot i$

by Pythia [14]). All the traces incur at least 1 LLC miss per kilo-instruction (MPKI) under the baseline. In total, there are 76 single-core workload traces.

For multi-core evaluation, we construct 50 heterogeneous workloads for each core count configuration using SPEC17 traces. In an n -core heterogeneous workload, we randomly select n traces, with one assigned to each core. If a core finishes execution early, it is replayed until all cores complete.

Prefetchers. Inspired by the prefetcher families integrated in AMD Zen 3 [4] and Arm Neoverse V2 [5], our system integrates four state-of-the-art prefetchers, each representing the leading approach in a distinct prefetching paradigm: (1) Berti [37], a local-delta-based prefetcher that learns timely and accurate stride deltas individually for each PC; (2) BOP [36], a global-delta-based prefetcher that identifies a single optimal delta shared across all PCs, ensuring timeliness; (3) PMP [24], a bit-pattern-based prefetcher that enhances the SMS by merging similar access bit patterns and accurately extracting prefetch targets from them; and (4) Prophet [30], a correlation-based prefetcher that builds upon Triage [56] and Triangel [7], leveraging profile-guided techniques to optimize metadata management and improve performance. All prefetchers are trained on L2 cache access stream.

Prefetcher Management Schemes. We compare I-POP against four prefetcher management schemes mentioned in Section II. Their detailed configurations are provided in Table III. For DOL, we assign priority orders based on the individually measured accuracy of each prefetcher. For Bandit, we configure $2^4 = 16$ arms, each representing a unique on/off combination of four prefetchers. For Alecto, we adopt the same configuration as the original paper [31]. For Sandbox [46], we set $K = 1$, meaning only the prefetcher with the highest score can issue prefetch requests in the next phase. For I-POP, we set T_{NoC} to 0 in the single-core system because the on-chip interconnect is not modeled in this configuration. DOL, Bandit, Sandbox, and I-POP use the default prefetch degree that yields their best performance (see Figure 19(c)): 4, 8, 4, and 2, respectively, and Alecto employs the complete dynamic degree adjustment mechanism in the original paper. In all our evaluations, these schemes refer solely to their management algorithms, independent of the specific composite prefetchers used in their original designs. For fairness, they operate on the same set of composite prefetchers as I-POP.

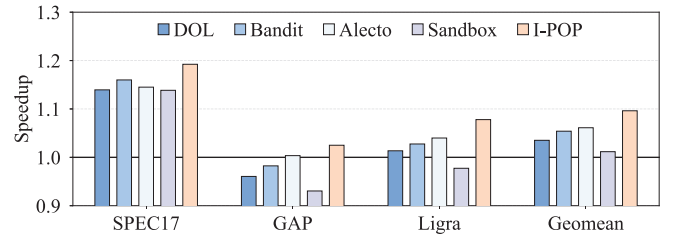


Fig. 11: IPC speedup in the single-core system.

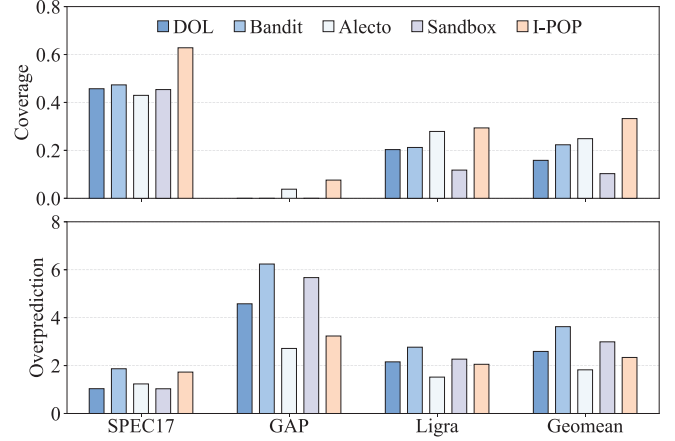


Fig. 12: Coverage and overprediction in the single-core system, normalized to L2 cache misses of the baseline.

VI. EVALUATION

A. Single-core Performance

Figure 11 shows the IPC speedup of five schemes in a single-core system. On average, I-POP improves performance by 9.7% over the baseline with a PC-Stride prefetcher and a basic throttling mechanism, outperforming DOL, Bandit, Alecto, and Sandbox by 6.1%, 4.2%, 3.5%, and 8.5%, respectively.

For SPEC17, all schemes deliver substantial improvements, with speedups no lower than 13.8%. I-POP further surpasses DOL by 5.3%, Bandit by 3.2%, Alecto by 4.8%, and Sandbox by 5.4%, demonstrating its ability to fully exploit the strengths of each prefetcher component. For GAP and Ligra, the performance gains are noticeably smaller than on SPEC17 because the underlying prefetchers struggle to capture the memory access patterns of certain graph workloads. On GAP, DOL, Bandit, and Sandbox even exhibit performance degradations of up to 7.0%, while Alecto provides only marginal improvements. In contrast, I-POP improves performance by 2.5%, outperforming DOL, Bandit, Alecto, and Sandbox by 6.4%, 3.2%, 4.7%, and 5.4%. On Ligra, I-POP achieves a 7.8% improvement, exceeding the performance of DOL, Bandit, Alecto, and Sandbox by 6.4%, 5.0%, 3.8%, and 10.1%. These results demonstrate I-POP's ability to promptly suppress harmful prefetchers and mitigate their negative impact on performance.

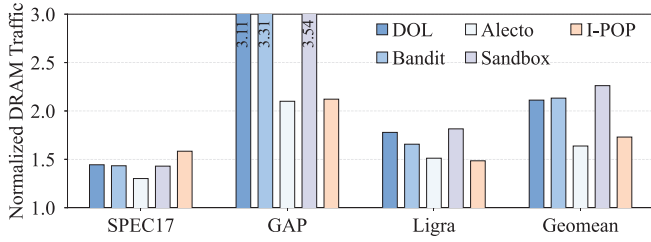


Fig. 13: DRAM traffic in the single-core system, normalized to the baseline.

B. Coverage & Overprediction

Figure 12 shows the coverage and overprediction of each scheme in the single-core system. Coverage denotes the fraction of reduced demand misses, and overprediction represents the fraction of useless prefetches, with both metrics normalized to the baseline's L2 cache misses.

Across all benchmark suites, I-POP delivers the highest coverage, with particularly strong gains on SPEC17 where it surpasses the other four schemes by 15.5% to 19.9%. On average, I-POP achieves 17.4%, 10.9%, 8.4%, and 23.0% higher coverage than DOL, Bandit, Alecto, and Sandbox, respectively. This advantage comes from I-POP's strategy of keeping prefetchers active as long as their PE values remain positive, maximizing prediction opportunities. The resulting increase in coverage is a key factor behind I-POP's superior performance gains. It is worth noting that on GAP, DOL, Bandit, and Sandbox exhibit negative coverage, indicating that they introduce more additional misses than they eliminate, ultimately degrading performance.

All schemes exhibit high overprediction rates, primarily because concurrently running multiple prefetchers amplifies the number of useless prefetches. Relative to DOL, Bandit, and Sandbox, I-POP shows 25.1%, 128.5%, and 65.2% lower overprediction on average, respectively. Although Alecto shows 51.6% lower overprediction than I-POP, this is due to its more restrictive prefetcher activation policies. Specifically, Alecto disables all prefetchers with accuracy below the PB threshold (0.75) once any prefetcher exceeds it. In contrast, I-POP often keeps multiple prefetchers active as long as their PE values are positive. While this results in some increase in overprediction, it does not increase DRAM traffic significantly (Section VI-C) and leads to improved coverage and overall performance.

C. DRAM Traffic

We define Normalized DRAM Traffic as the ratio of total DRAM accesses relative to the baseline and present the results in Figure 13. On average, I-POP incurs 38.2%, 40.2%, and 53.1% less DRAM traffic than DOL, Bandit, and Sandbox, respectively, while generating 9.2% more traffic than Alecto, primarily driven by its behaviors on SPEC17. On GAP and Ligra, however, I-POP produces DRAM traffic comparable to or even lower than Alecto, as it promptly disables prefetchers that harm performance. This shows that I-POP achieves higher coverage and IPC speedup without introducing additional

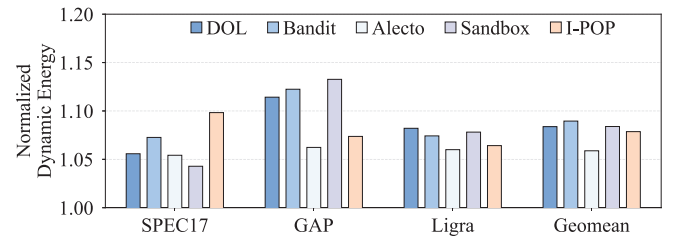


Fig. 14: Dynamic energy consumption in the single-core system, normalized to the baseline.

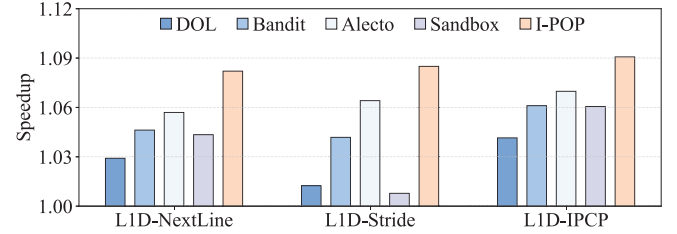


Fig. 15: IPC speedup across all workloads when L1D is equipped with a prefetcher.

DRAM pressure. Notably, DOL, Bandit, and Sandbox generate substantially higher DRAM traffic on GAP than both Alecto and I-POP, as they lack an effective negative-feedback mechanism and continue to tolerate numerous useless prefetches.

D. Energy Efficiency

Figure 14 presents the dynamic energy consumption of five schemes under the 2400 MT/s configuration, normalized to the baseline. On average, I-POP consumes 0.5%, 1.1%, and 0.5% less energy than DOL, Bandit, and Sandbox, respectively, and 2.0% more than Alecto. On SPEC17, I-POP consumes more energy because composite prefetchers are generally effective on these workloads and tend to yield positive PE values, which causes I-POP to keep them enabled more frequently, increasing prefetch requests and energy usage. On GAP, I-POP and Alecto consume substantially less energy than the other three schemes, with a maximum difference of 7.0%, which aligns with the above observations on overprediction and DRAM traffic.

E. Multi-level Prefetching

Most processors implement prefetchers at multiple cache levels. Figure 15 presents the performance of five schemes under scenarios where the L1D cache is equipped with a Next-line, PC-Stride, or IPCP [42] prefetcher. In all three cases, I-POP consistently outperforms all other schemes across all benchmark suites. On average, I-POP achieves performance improvements of 2.5%, 2.1%, and 2.1% over the second-best scheme, Alecto, under Next-line, Stride, and IPCP configurations, respectively. These results demonstrate that I-POP effectively manages L2 prefetchers even in the presence of L1 prefetcher noise.

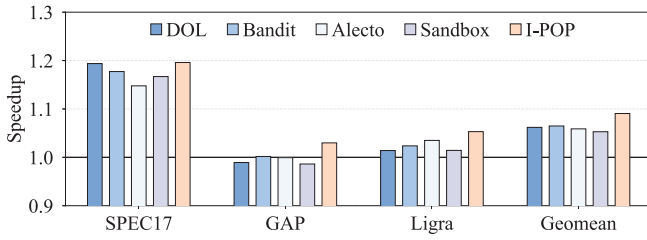


Fig. 16: IPC speedup in the single-core system with Berti and PMP in L1D cache, and BOP and Prophet in L2 cache.

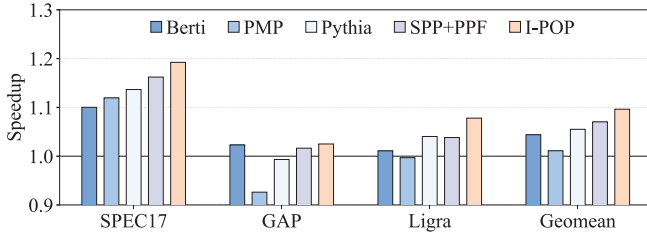


Fig. 17: IPC speedup of I-POP and non-composite prefetchers.

We also evaluate a multi-level, multi-prefetcher configuration with Berti and PMP at L1D cache, and BOP and Prophet at L2 cache. All control schemes are implemented in each cache to manage its resident prefetchers. As shown in [Figure 16](#), on average, I-POP outperforms DOL, Bandit, Alecto and Sandbox by 2.9%, 2.6%, 3.2% and 3.8%, respectively. The performance gaps are smaller than in the single-level setting because partitioning prefetchers to two cache levels reduces each controller’s state/action space, leading the schemes to make more similar decisions.

F. Comparing to Non-composite Prefetchers

To highlight the benefits of composite prefetching, we compare the IPC speedup of I-POP with four state-of-the-art non-composite L2 prefetchers: Berti [37], PMP [24], Pythia [14], and SPP+PPF [16, 28]. Pythia is a state-of-the-art RL-based prefetcher that leverages various types of program context information to predict address deltas. SPP+PPF enhances the delta-based SPP prefetcher with a perceptron-based prefetch filter to suppress inaccurate prefetches. As shown in [Figure 17](#), I-POP outperforms Berti, PMP, Pythia and SPP+PPF by 5.2%, 8.5%, 4.1% and 2.6%, respectively. These results further demonstrate the advantage of composite prefetching, which leverages a diverse set of prefetchers to broaden the range of recognizable access patterns and deliver superior performance.

G. Managing Diverse Prefetcher Combinations

To further evaluate I-POP’s performance when managing diverse prefetcher combinations, we consider a broader set of configurations. We construct a prefetcher pool containing eight representative prefetchers, grouped into four pairs based on their characteristics: (Berti (B), SPP+PPF (F)), (BOP (O), MLOP (M) [48]), (PMP (P), Bingo (G) [11]), and (Prophet (R), Domino (D)[10]). By selecting one prefetcher from each

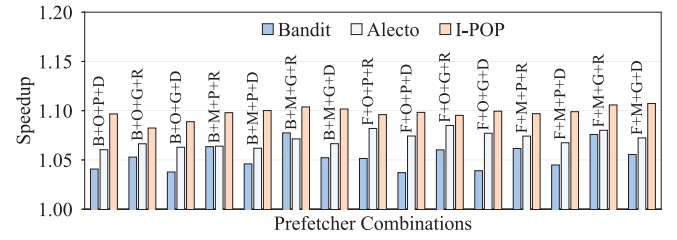


Fig. 18: IPC speedup of three schemes when managing various prefetcher combinations. B: Berti, F: SPP+PPF, O: BOP, M: MLOP, P: PMP, G: Bingo, R: Prophet, D: Domino.

group, we generate 16 unique combinations. [Figure 18](#) shows the average speedup of three schemes of 16 combinations across all workloads. I-POP consistently outperforms both Bandit and Alecto across all combinations, with an average improvement of 4.4% and 2.6%, respectively.

H. Sensitivity Studies

DRAM bandwidths. [Figure 19\(a\)](#) illustrates the geometric mean of IPC speedup over the baseline as DRAM bandwidth scales from 1200 MT/s to 6000 MT/s. We observe that I-POP consistently delivers the highest performance across all DRAM bandwidth configurations. At 1200 MT/s, I-POP outperforms DOL, Bandit, Alecto, and Sandbox by 5.4%, 4.4%, 3.0%, and 7.6%, respectively. As memory bandwidth increases, I-POP maintains a larger lead of about 3.5% over the second-best approach. This advantage stems from I-POP’s ability to account for the impact of each prefetch request on DRAM latency and to dynamically adjust prefetcher aggressiveness accordingly. As a result, I-POP adapts effectively across a wide range of DRAM bandwidth conditions.

L2 cache size. [Figure 19\(b\)](#) presents the IPC speedup across four benchmark suites as the L2 cache size varies from 128 KB to 2048 KB. I-POP consistently outperforms other schemes across all configurations. At the 1024 KB configuration, I-POP surpasses DOL, Bandit, Alecto, and Sandbox by 7.0%, 5.1%, 4.1%, and 8.9%, respectively. As the L2 cache size decreases, the performance advantage of I-POP over other schemes progressively increases. At the 128 KB configuration, I-POP outperforms DOL, Bandit, Alecto, and Sandbox by 8.3%, 5.5%, 6.1%, and 8.2%, respectively. This trend is attributed to the increasing impact of cache pollution under smaller cache configurations. By effectively capturing the effects of cache pollution, I-POP demonstrates strong adaptability across a wide range of cache sizes.

Prefetching degrees. [Figure 19\(c\)](#) shows the IPC speedup of four approaches across various default prefetch degrees. We do not evaluate Alecto because its built-in degree-tuning mechanism renders it insensitive to the default prefetch degree. The default degree determines the initial aggressiveness of each prefetcher. I-POP then dynamically adjusts this degree at runtime based on PE values and average DRAM latency, as described in [Section IV-B](#). Across all configurations, I-POP consistently achieves the highest performance. When each

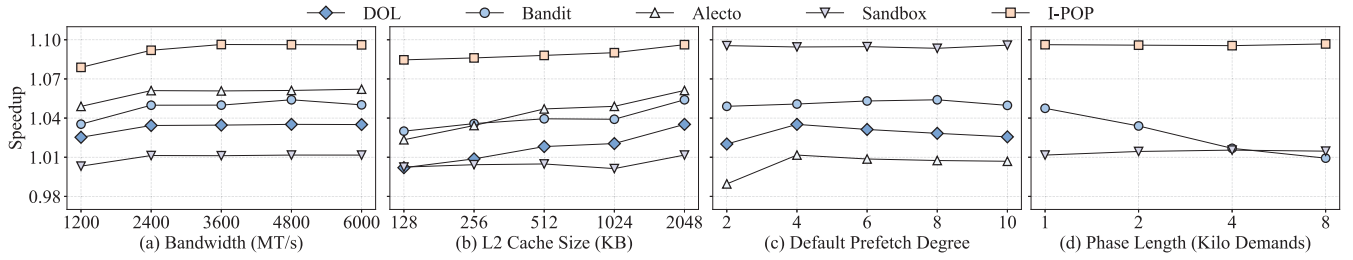


Fig. 19: IPC speedup of prefetcher management schemes with varying (a) DRAM bandwidth, (b) L2 cache size, (c) default prefetch degree, and (d) phase length.

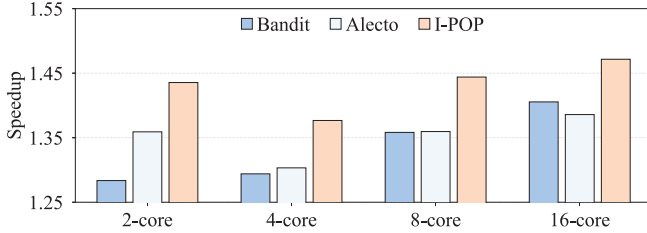


Fig. 20: IPC speedup in multi-core systems.

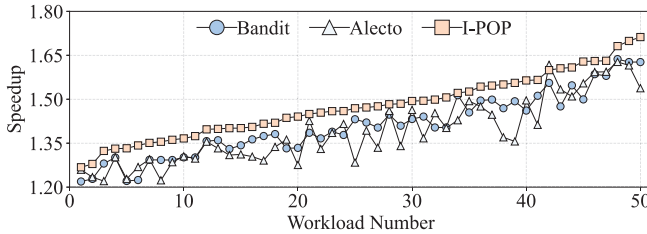


Fig. 21: Speedup on 16-core workloads, sorted by I-POP.

method uses its best-performing configuration, I-POP outperforms DOL, Bandit, and Sandbox by 5.9%, 3.9%, and 8.3%, respectively. Thanks to its effective dynamic aggressiveness adjustment mechanism, I-POP exhibits minimal sensitivity to the choice of default degree.

Phase length. Bandit, Sandbox, and I-POP manage all prefetchers at the phase granularity measured by L2 demand accesses. Figure 19(d) presents their performance as the phase length increases from 1K to 8K. I-POP consistently outperforms Bandit by 4.9%, 6.2%, 7.9% and 8.8%, respectively. As the phase length increases, Bandit's performance gradually declines, showing a performance gap of up to 3.9%. This degradation stems from Bandit's adjustment lag (see Section II-D), which lasts several phases. Longer phases introduce greater adjustment lag and thus larger performance losses, whereas shorter phases reduce the lag but incur higher, potentially prohibitive, computational overhead.

I. Multi-core Performance

Figure 20 compares the performance improvement of I-POP with two state-of-the-art approaches, Bandit and Alecto, in multi-core systems ranging from 2 to 16 cores. I-POP

TABLE IV: Storage overhead of I-POP with $P = 4$

Structure	Description (bit)	Storage
I-POP	# Prefetch Holding Table (512-entry) • $512 \times (6 + 1 + P)$	1416 B
	# Pollution Holding Table (512-entry) • $512 \times (6 + P)$	
	# Registers • Private: 7 16-bit, 1 4-bit • Shared: 3 16-bit, 2 32-bit • $116 \times P + 112$	
MSHR	# Extension Fields of 32-entry • $32 \times (16 + 1 + P)$	84 B
Total		1.46 KB

consistently outperforms both Bandit and Alecto across all system configurations. In 8-core and 16-core systems, I-POP achieves 8.6% and 6.6% higher performance than Bandit, and 8.4% and 8.6% higher than Alecto, respectively. Figure 21 provides a comprehensive performance comparison of the three schemes across all 16-core workloads. I-POP consistently achieves the highest performance in all but one workload, outperforming the second-best scheme by up to 20.0%. These results demonstrate the superior scalability of I-POP, highlighting its effectiveness in managing multiple prefetchers even in complex multi-core systems.

J. Storage Overhead

Table IV summarizes the storage overhead of I-POP. Both PfHT and PoHT are implemented as 512-entry direct-mapped lookup tables with 6-bit tags, consistent with the configuration used in Alecto. Our experiments demonstrate that this configuration is sufficient for high performance, and increasing the storage budget offers negligible performance gains. Each prefetcher requires a 4-bit state register and seven 16-bit event counters. In addition, I-POP uses four shared registers to compute L_{LLC} and L_{DRAM} , along with one counter for phase partitioning. With four prefetchers, I-POP incurs only 1.46 KB of storage overhead per core, which is just 27.7% of the 5.27 KB required by Alecto. Moreover, each additional prefetcher introduces only 146.5 B of overhead in I-POP, compared with 1184 B in Alecto, highlighting I-POP's superior scalability.

VII. RELATED WORK

Prefetch Throttling Mechanisms. Existing prefetch throttling techniques [8, 18, 22, 43, 45, 52, 53] typically define

multiple aggressiveness levels and rely primarily on prefetch accuracy to guide throttling decisions. For example, FDP [53] performs local throttling based on a combination of prefetch accuracy, lateness, and cache pollution. HPAC [18] extends FDP by incorporating information from other cores to enable global coordination. Both are qualitative and threshold-based methods (i.e., compare to fixed thresholds). However, no one threshold is universally suitable. I-POP uses quantitative metrics to make decisions. Additionally, some techniques tailor the control objective: SPAC [43] partitions prefetchers into aggressive and meek groups and explores five throttling combinations to optimize fair-speedup [35] of multi-core systems.

Notably, some techniques adopt utility-driven control, which is related in spirit to I-POP. UDP [41] uses a Bloom filter to capture the utility of instruction prefetches, improving FDIP [47]. CAFFEINE [45] introduces a utility metric for each core to support global throttling. However, their utility metrics differ fundamentally from I-POP's PE. UDP essentially aims to identify useful prefetches, and CAFFEINE's metric is considerably more complex than PE and requires substantial modifications to memory controllers to track DRAM-level events. In contrast, PE is a lightweight, latency-centric metric that quantifies the net utility of prefetching.

A complementary axis to latency is load criticality [12, 27, 34, 40, 44]. CLIP [44] identifies loads that frequently stall at the head of the reorder buffer (ROB) and throttles prefetching based on both criticality and accuracy. This perspective also helps explain why no single accuracy threshold is universally appropriate: a prefetch's utility depends not only on the latency it hides but also on whether it serves ROB-head-critical loads that would otherwise stall. PE currently captures latency-centric utility; as future work, we plan to augment PE with criticality awareness to better reflect the system-level performance impact of prefetches.

Moreover, all techniques above primarily target systems with a single prefetcher per core. In contrast, I-POP is designed for managing multiple prefetchers within a core, introducing a novel metric to quantify each prefetcher's impact on performance with minimal implementation overhead.

Hardware Prefetching. Previous hardware prefetchers can be broadly classified into two types. (1) Temporal prefetchers [7, 10, 23, 30, 39, 54, 56, 57] track full block addresses of memory accesses and replay historical address sequences to issue prefetches. GHB [39], STMS [54], ISB [23], Domino [10], and MISB [57] rely on global history tables and store a large amount of metadata off-chip, which requires frequent metadata accesses. Triage [56] identifies the most critical metadata and stores it in a portion of the LLC. Triangle [7] extends Triage by introducing sampling-based techniques to evaluate metadata quality, improving the timeliness and accuracy of prefetching.

(2) Spatial prefetchers [11, 14, 15, 17, 19, 24, 28, 33, 36, 37, 38, 49, 51] capture spatial correlations among memory accesses within different memory regions, including both address deltas and access footprints (represented as bit-patterns). SPP [28] creates signatures for address sequences and uses them to predict the next delta. VLDP [49] employs multiple

delta-prediction tables indexed by history delta sequences of varying lengths. DSPatch [15] constructs two types of bit-patterns using AND and OR operations and dynamically selects between them based on DRAM bandwidth usage. PARS [33] learns bit-patterns across multiple region sizes simultaneously, improving performance while reducing storage overhead.

These prior efforts focus on improving the prefetching algorithms and hardware designs of individual prefetchers. In contrast, I-POP is a prefetcher management scheme for multi-prefetcher systems, dynamically adjusting the activation and aggressiveness of each prefetcher.

VIII. CONCLUSION

We present I-POP, a high-performance and low-overhead prefetcher management scheme for multi-prefetcher systems. I-POP introduces a novel runtime metric, PE, which aggregates each prefetch request's beneficial and harmful effects to precisely quantify the performance impact of prefetching, effectively overcoming the limitations of prior metrics. I-POP consists of a Metric Collector and a Control Engine to compute PE values and dynamically manage prefetchers accordingly. Our evaluations show that PE achieves a stronger positive correlation with IPC performance than prior metrics. Benefiting from PE's accurate and clear reflection of a prefetcher's impact on IPC performance, I-POP effectively manages multiple prefetchers and achieves greater performance improvements over state-of-the-art approaches across a wide range of workloads and system configurations, while incurring only 1.46 KB of storage overhead.

ACKNOWLEDGEMENTS

We would like to thank all the anonymous reviewers for their insightful comments and suggestions. This work is supported in part by the Key Research and Development Program of Zhejiang Province under Grant No. 2025C02103, in part by the National Natural Science Foundation of China under Grant No. 92373205, and in part by the Alibaba Innovative Research Program.

REFERENCES

- [1] "SPEC CPU 2017," <https://www.spec.org/cpu2017>, 2006, accessed: 2025-07-31.
- [2] "Micron dram power calculator," <https://www.micron.com/content/dam/micron/global/secure/products/technical-note/dram/tn4007-ddr4-power-calculation.pdf>, 2015, accessed: 2025-07-31.
- [3] "The 3rd Data Prefetching Championship," <https://dpc3.compas.cs.stonybrook.edu>, 2019, accessed: 2025-07-31.
- [4] "Software Optimization Guide for AMD EPYC™ 7003 Processors," https://ebrandbuilders.com/wp-content/uploads/2022/04/56665_3.00.pdf, 2020, accessed: 2025-07-31.
- [5] "Arm Neoverse V2 platform: Leadership Performance and Power Efficiency for NextGeneration Cloud Computing, ML and HPC Workloads,"

- <https://hc2023.hotchips.org/assets/program/conference/day1/CPU1/HC2023.Arm.MagnusBruce.v04.FINAL.pdf>, 2023, accessed: 2025-07-31.
- [6] “Hardware Prefetch Controls for Intel® Atom® Cores,” <https://www.intel.com/content/www/us/en/content-details/795247/hardware-prefetch-controls-for-intel-atom-cores.html>, 2023, accessed: 2025-07-31.
 - [7] S. Ainsworth and L. Mukhanov, “Triangel: A high-performance, accurate, timely on-chip temporal prefetcher,” in *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE/ACM, 2024, pp. 1202–1216.
 - [8] J. Albericio, R. Gran, P. Ibáñez, V. Viñals, and J. M. Llabería, “Abs: A low-cost adaptive controller for prefetching in a banked shared last-level cache,” *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 8, no. 4, pp. 1–20, 2012.
 - [9] G. Ayers, H. Litz, C. Kozyrakis, and P. Ranganathan, “Classifying memory access patterns for prefetching,” in *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2020, pp. 513–526.
 - [10] M. Bakhshalipour, P. Lotfi-Kamran, and H. Sarbazi-Azad, “Domino temporal data prefetcher,” in *Proceedings of the 24th Annual International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2018, pp. 131–142.
 - [11] M. Bakhshalipour, M. Shakerinava, P. Lotfi-Kamran, and H. Sarbazi-Azad, “Bingo spatial data prefetcher,” in *Proceedings of the 25th Annual International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2019, pp. 399–411.
 - [12] S. Bandishte, J. Gaur, Z. Sperber, L. Rappoport, A. Yoaz, and S. Subramoney, “Focused value prediction: Concepts, techniques and implementations presented in this paper are subject matter of pending patent applications, which have been filed by intel corporation,” in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE/ACM, 2020, pp. 79–91.
 - [13] S. Beamer, K. Asanović, and D. Patterson, “The gap benchmark suite,” *arXiv preprint arXiv:1508.03619*, 2015.
 - [14] R. Bera, K. Kanellopoulos, A. Nori, T. Shahroodi, S. Subramoney, and O. Mutlu, “Pythia: A customizable hardware prefetching framework using online reinforcement learning,” in *Proceedings of the 54th Annual International Symposium on Microarchitecture (MICRO)*. IEEE/ACM, 2021, pp. 1121–1137.
 - [15] R. Bera, A. V. Nori, O. Mutlu, and S. Subramoney, “Dspatch: Dual spatial pattern prefetcher,” in *Proceedings of the 52nd Annual International Symposium on Microarchitecture (MICRO)*. IEEE/ACM, 2019, pp. 531–544.
 - [16] E. Bhatia, G. Chacon, S. Pugsley, E. Teran, P. V. Gratz, and D. A. Jiménez, “Perceptron-based prefetch filtering,” in *Proceedings of the 46th International Symposium on Computer Architecture (ISCA)*. IEEE/ACM, 2019, pp. 1–13.
 - [17] T.-F. Chen and J.-L. Baer, “Effective hardware-based data prefetching for high-performance processors,” *IEEE Transactions on Computers (TC)*, vol. 44, no. 5, pp. 609–623, 1995.
 - [18] E. Ebrahimi, O. Mutlu, C. J. Lee, and Y. N. Patt, “Coordinated control of multiple prefetchers in multi-core systems,” in *Proceedings of the 42nd Annual International Symposium on Microarchitecture (MICRO)*. IEEE/ACM, 2009, pp. 316–326.
 - [19] J. W. Fu, J. H. Patel, and B. L. Janssens, “Stride directed prefetching in scalar processors,” in *Proceedings of the 25th Annual International Symposium on Microarchitecture (MICRO)*. IEEE/ACM, 1992, pp. 102–110.
 - [20] G. Gerogiannis and J. Torrellas, “Micro-armed bandit: Lightweight & reusable reinforcement learning for microarchitecture decision-making,” in *Proceedings of the 56th Annual International Symposium on Microarchitecture (MICRO)*. IEEE/ACM, 2023, pp. 698–713.
 - [21] N. Gober, G. Chacon, L. Wang, P. V. Gratz, D. A. Jimenez, E. Teran, S. Pugsley, and J. Kim, “The championship simulator: Architectural simulation for education and competition,” *arXiv preprint arXiv:2210.14324*, 2022.
 - [22] W. Heirman, K. D. Bois, Y. Vandriessche, S. Eyeraman, and I. Hur, “Near-side prefetch throttling: Adaptive prefetching for high-performance many-core processors,” in *Proceedings of the 27th international conference on parallel architectures and compilation techniques (PACT)*, 2018, pp. 1–11.
 - [23] A. Jain and C. Lin, “Linearizing irregular memory accesses for improved correlated prefetching,” in *Proceedings of the 46th Annual International Symposium on Microarchitecture (MICRO)*. IEEE/ACM, 2013, pp. 247–259.
 - [24] S. Jiang, Q. Yang, and Y. Ci, “Merging similar patterns for hardware prefetching,” in *Proceedings of the 55th Annual International Symposium on Microarchitecture (MICRO)*. IEEE/ACM, 2022, pp. 1012–1026.
 - [25] D. A. Jiménez and C. Lin, “Dynamic branch prediction with perceptrons,” in *Proceedings of the 17th Annual International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2001, pp. 197–206.
 - [26] D. Joseph and D. Grunwald, “Prefetching using markov predictors,” in *Proceedings of the 24th Annual International Symposium on Computer Architecture (ISCA)*. IEEE/ACM, 1997, pp. 252–263.
 - [27] N. S. Kalani and B. Panda, “Instruction criticality based energy-efficient hardware data prefetching,” *IEEE Computer Architecture Letters*, vol. 20, no. 2, pp. 146–149, 2021.
 - [28] J. Kim, S. H. Pugsley, P. V. Gratz, A. N. Reddy, C. Wilkerson, and Z. Chishti, “Path confidence based looka-

- head prefetching,” in *Proceedings of the 49th Annual International Symposium on Microarchitecture (MICRO)*. IEEE/ACM, 2016, pp. 1–12.
- [29] S. Kondguli and M. Huang, “Division of labor: A more effective approach to prefetching,” in *Proceedings of the 45th Annual International Symposium on Computer Architecture (ISCA)*. IEEE/ACM, 2018, pp. 83–95.
- [30] M. Li, Q. Zhang, Y. Gao, W. Fang, Y. Lu, Y. Ren, and Z. Xie, “Profile-guided temporal prefetching,” in *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA)*. IEEE/ACM, 2025, p. 572–585.
- [31] M. Li, Q. Zhang, Y. Ren, and Z. Xie, “Integrating prefetcher selection with dynamic request allocation improves prefetching efficiency,” in *Proceedings of the 31th Annual International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2025.
- [32] S. Li, K. Chen, J. H. Ahn, J. B. Brockman, and N. P. Jouppi, “Cacti-p: Architecture-level modeling for sram-based structures with advanced leakage reduction techniques,” in *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 2011, pp. 694–701.
- [33] Y. Lin, W. Lin, J. Xu, Y. Chen, Z. Jin, J. Qin, J. He, S. Cai, Y. Zhang, Z. Wang *et al.*, “Pars: A pattern-aware spatial data prefetcher supporting multiple region sizes,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, vol. 43, no. 11, pp. 3638–3649, 2024.
- [34] H. Litz, G. Ayers, and P. Ranganathan, “Crisp: critical slice prefetching,” in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2022, pp. 300–313.
- [35] K. Luo, J. Gummaraju, and M. Franklin, “Balancing throughput and fairness in smt processors,” in *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE Computer Society, 2001, pp. 164–165.
- [36] P. Michaud, “Best-offset hardware prefetching,” in *Proceedings of the 22nd Annual International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2016, pp. 469–480.
- [37] A. Navarro-Torres, B. Panda, J. Alastruey-Benedé, P. Ibáñez, V. Viñals-Yúfera, and A. Ros, “Berti: an accurate local-delta data prefetcher,” in *Proceedings of the 55th Annual International Symposium on Microarchitecture (MICRO)*. IEEE/ACM, 2022, pp. 975–991.
- [38] A. Navarro-Torres, B. Panda, J. Alastruey-Benedé, P. Ibáñez, V. Viñals-Yúfera, and A. Ros, “A complexity-effective local delta prefetcher,” *IEEE Transactions on Computers*, 2025.
- [39] K. J. Nesbit and J. E. Smith, “Data cache prefetching using a global history buffer,” in *Proceedings of the 10th Annual International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2004, pp. 96–96.
- [40] A. V. Nori, J. Gaur, S. Rai, S. Subramoney, and H. Wang, “Criticality aware tiered cache hierarchy: A fundamental relook at multi-level cache hierarchies,” in *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. IEEE/ACM, 2018, pp. 96–109.
- [41] S. Oh, M. Xu, T. A. Khan, B. Kasikci, and H. Litz, “Udp: Utility-driven fetch directed instruction prefetching,” in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE/ACM, 2024, pp. 1188–1201.
- [42] S. Pakalapati and B. Panda, “Bouquet of instruction pointers: Instruction pointer classifier-based spatial hardware prefetching,” in *Proceedings of the 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE/ACM, 2020, pp. 118–131.
- [43] B. Panda, “Spac: A synergistic prefetcher aggressiveness controller for multi-core systems,” *IEEE Transactions on Computers (TC)*, vol. 65, no. 12, pp. 3740–3753, 2016.
- [44] B. Panda, “Clip: Load criticality based data prefetching for bandwidth-constrained many-core systems,” in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture (ISCA)*. IEEE/ACM, 2023, pp. 714–727.
- [45] B. Panda and S. Balachandran, “Caffeine: A utility-driven prefetcher aggressiveness engine for multicores,” *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 12, no. 3, pp. 1–25, 2015.
- [46] S. H. Pugsley, Z. Chishti, C. Wilkerson, P.-f. Chuang, R. L. Scott, A. Jaleel, S.-L. Lu, K. Chow, and R. Balasubramanian, “Sandbox prefetching: Safe run-time evaluation of aggressive prefetchers,” in *Proceedings of the 20th Annual International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2014, pp. 626–637.
- [47] G. Reinman, B. Calder, and T. Austin, “Fetch directed instruction prefetching,” in *Proceedings of the 32nd Annual International Symposium on Microarchitecture (MICRO)*. IEEE/ACM, 1999, pp. 16–27.
- [48] M. Shakerinava, M. Bakhshalipour, P. Lotfi-Kamran, and H. Sarbazi-Azad, “Multi-lookahead offset prefetching,” *The Third Data Prefetching Championship*, 2019.
- [49] M. Shevgoor, S. Koladiya, R. Balasubramanian, C. Wilkerson, S. H. Pugsley, and Z. Chishti, “Efficiently prefetching complex address patterns,” in *Proceedings of the 48th Annual International Symposium on Microarchitecture (MICRO)*. IEEE/ACM, 2015, pp. 141–152.
- [50] J. Shun and G. E. Blelloch, “Ligra: a lightweight graph processing framework for shared memory,” in *Proceedings of the 18th SIGPLAN Symposium on Principles and Practice of Parallel Programming*. ACM, 2013, pp. 135–146.
- [51] S. Somogyi, T. F. Wenisch, A. Ailamaki, B. Falsafi, and A. Moshovos, “Spatial memory streaming,” in *Proceedings of the 33rd Annual International Symposium on*

Computer Architecture (ISCA). IEEE/ACM, 2006, pp. 252–263.

- [52] A. Sridharan, B. Panda, and A. Sez nec, “Band-pass prefetching: an effective prefetch management mechanism using prefetch-fraction metric in multi-core systems,” *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 14, no. 2, pp. 1–27, 2017.
- [53] S. Srinath, O. Mutlu, H. Kim, and Y. N. Patt, “Feed-back directed prefetching: Improving the performance and bandwidth-efficiency of hardware prefetchers,” in *Proceedings of the 13rd Annual International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2007, pp. 63–74.
- [54] T. F. Wenisch, M. Ferdman, A. Ailamaki, B. Falsafi, and A. Moshovos, “Practical off-chip meta-data for temporal memory streaming,” in *Proceedings of the 15th Annual International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2009, pp. 79–90.
- [55] C.-J. Wu, A. Jaleel, W. Hasenplaugh, M. Martonosi, S. C. Steely Jr, and J. Emer, “Ship: Signature-based hit predictor for high performance caching,” in *Proceedings of the 44th Annual International Symposium on Microarchitecture (MICRO)*. IEEE/ACM, 2011, pp. 430–441.
- [56] H. Wu, K. Nathella, J. Pusdesris, D. Sunwoo, A. Jain, and C. Lin, “Temporal prefetching without the off-chip meta-data,” in *Proceedings of the 52nd Annual International Symposium on Microarchitecture (MICRO)*. IEEE/ACM, 2019, pp. 996–1008.
- [57] H. Wu, K. Nathella, D. Sunwoo, A. Jain, and C. Lin, “Efficient metadata management for irregular data prefetching,” in *Proceedings of the 46th Annual International Symposium on Computer Architecture (ISCA)*. IEEE/ACM, 2019, pp. 449–461.
- [58] P. Zhang, R. Kannan, A. Srivastava, A. V. Nori, and V. K. Prasanna, “Resemble: reinforced ensemble framework for data prefetching,” in *Proceedings of the 34th Annual International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. IEEE, 2022, pp. 1–14.